

Spring & SpringBoot — Quick Revision Notes

1. How Web Apps Communicate

The browser (client) and the server run on different machines and need a way to talk to each other — this is the foundation of web development.

2. Client-Server Architecture

Client = the side that asks for something (browser, mobile app, Postman). **Server** = the side that receives the request, processes it, and sends back a response (Amazon server, Spring Boot app).

3. HTTP: The Language of the Web

HTTP (HyperText Transfer Protocol) is the rulebook that defines how a request/response should look — method, URL, data, and status code.

4. Request-Response Cycle

The pattern is always the same: Client sends a request → Server processes it → Server sends a response → Client uses/displays it.

5. Anatomy of an HTTP Request

A request has 4 main parts: **Method** (GET/POST/PUT/PATCH/DELETE), **URL**, **Headers**, **Body**. GET = read, POST = create, PUT = full update, PATCH = partial update, DELETE = remove.

6. Anatomy of an HTTP Response

A response contains: **Status Code** (200 = success, 404 = not found, 500 = server error), **Headers**, and **Body**.

7. JVM (Java Virtual Machine)

A `.java` file is compiled by `javac` into bytecode (`.class`), which the JVM then runs — this is why Java is "write once, run anywhere."

8. Normal Java Program vs Web Application

A normal Java program starts, runs, finishes, and exits. A web application keeps running continuously, waiting for and processing requests.

9. Core Java Alone Is Not Enough

Core Java understands classes, threads, and collections, but it doesn't automatically understand web concepts like HTTP requests, URLs, headers, or cookies.

10. Can Java Open Network Connections?

Yes, using the `java.net` package (like `ServerSocket`), Java can listen on a network — but it only receives raw TCP bytes, which still need to be manually interpreted as HTTP.

11. Manual Work Needed in Core Java

Without a framework, you'd have to manually open a port, parse requests, write routing logic, build responses, and manage threads — a lot of repetitive boilerplate.

12. The Problem Before Servlets

Something was needed to translate an incoming HTTP request into an actual Java method call — this gap is what Servlets solved.

13. Java Servlets

A Servlet is a special Java class designed to handle HTTP requests. It runs inside a **Servlet Container** (like Tomcat, Jetty, or Undertow).

14. What Does a Servlet Container Do?

The container opens the port, parses HTTP requests, creates request/response objects, manages threads, and calls the servlet's `doGet()`/`doPost()` methods.

15. Why Was Spring Needed?

Servlets worked, but large applications built directly with them had too much boilerplate, tight coupling, and difficult testing/maintenance — so the Spring Framework was created.

16. What is Spring Framework?

Spring makes enterprise Java development easier by introducing concepts like IoC, Dependency Injection, Bean Management, Configuration, and Loose Coupling.

17. Spring is an Ecosystem

Spring isn't a single library — it's a whole ecosystem of projects: Spring Core, MVC, Data, Security, AOP, Boot, and AI.

18. Spring Core

The foundation of Spring — provides IoC, Dependency Injection, Bean Management, and `ApplicationContext`. All other Spring modules are built on top of it.

19. Spring MVC

Built on top of Servlets + Spring Core — used to build web apps/REST APIs. Annotations (like `@GetMapping`) let you write clean controller code instead of raw servlet code.

20. Spring Data (JPA & Hibernate)

JDBC required writing SQL manually. Hibernate (an ORM) maps Java objects to database tables. **JPA** = the specification (what should be done), **Hibernate** = the implementation (how it's actually done). Spring Data JPA reduces the boilerplate even further.

21. Spring Security

Handles authentication and authorization — login, JWT, OAuth, roles, permissions, password encoding, and CSRF protection.

22. Spring AOP (Aspect-Oriented Programming)

Separates cross-cutting concerns (logging, security checks, transaction management) from the main business logic.

23. Spring AI

A newer part of the Spring ecosystem that helps integrate AI features (OpenAI, Gemini, Anthropic, vector databases, RAG, embeddings) into Java apps.

24. What is Spring Boot?

Spring Boot isn't a replacement for Spring — it's an **automation layer** on top of it, providing auto-configuration, starter dependencies, and embedded servers to speed up development.

25. Spring Boot is Opinionated

Spring Boot makes sensible default assumptions ("opinions") — e.g., adding a web dependency auto-configures Tomcat + MVC — so you don't need to configure everything manually.

26. Spring Boot vs Spring Framework

Spring Framework = core features + flexibility, more manual config. **Spring Boot** = built on top of Spring, auto-config + fast setup. Spring Boot uses Spring, it doesn't replace it.

27. Where Do Microservices Fit?

Microservices aren't a Spring module — they're an **architecture style** of splitting an app into small, independent services. Spring Boot is commonly used to build them.

28. Complete Flow: Browser to Spring Boot

Browser → Servlet Container → Servlet → Spring MVC → Spring Core → Spring Boot auto-config → Developer just writes the business logic.

29. Why This History Matters

Jumping straight into Spring Boot makes the code "work," but it feels like magic. Understanding the full journey (Client-Server → HTTP → JVM → Servlets → Spring → Spring Boot) makes everything make sense.

30. Final Summary (Evolution Chain)

Client-Server Architecture → HTTP → Limitation of Core Java → Java networking (sockets) → Manual HTTP parsing problem → Servlets/Servlet Container → Spring Framework → Spring ecosystem → Spring Boot.